

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf\_09101709-554\agent-ae2b06a53805091a6.jsonl`
- SHA-256: `d1c880018563f76736bf5767be21953bf6b4e4ba2517bab216b49eb92e9ba805`
- Source modified: `2026-06-25T17:13:15+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_09101709-554`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`

Transcript

1. user (2026-06-25T17:12:32.293Z)

Adversarially verify this PR-review finding for PR #381 (branch chore/nightly-quality-regression checked out; read .github/workflows/nightly.yml + ci.yml). Be skeptical ? default to INVALID if wrong, already-handled, or a non-issue.

Finding [LOW] Two offline ci.yml lanes are NOT mirrored (import-smoke, leak-scan) ? partial offline coverage, not a bug but worth noting/justifying
Location: .github/workflows/nightly.yml (whole file) vs .github/workflows/ci.yml:37-46 (leak-scan), :69-80 (import-smoke)
Detail: nightly.yml mirrors 4 of ci.yml's offline lanes verbatim (lint, typecheck, full-suite, golden-fixtures?golden-crossos). The file's header (L8-12) claims the lanes 'mirror ci.yml's offline jobs', but ci.yml has TWO more lanes that are equally offline and are NOT carried over:
- `leak-scan` (ci.yml:37-46): `python -m tools.leak\_scan` ? pure-stdlib PII/attribution guard, fully offline, no GPU. This is exactly the kind of structural rot a nightly default-branch guard is meant to catch.
- `import-smoke` (ci.yml:69-80): `python -m pytest tests/test\_import\_smoke.py` ? the explicit guard against the 7fbb0 SyntaxError/import-error class of breakage, deliberately offline (no cv2/torch/numpy).
(The other two omitted lanes ? `license-scan` ci.yml:146 and `build-artifact` ci.yml:172 ? are also offline; license-scan in particular guards the non-negotiable MIT/Apache/BSD licensing rule and would catch transitive-dep license drift, which is precisely the 'dependency / transitive-dep drift' the nightly header L4-5 says it exists to catch.)
This is a fidelity gap, not a runnability defect: the 4 included lanes are correct. But the header overstates coverage ('mirror ci.yml's offline jobs verbatim') while silently dropping offline jobs that fit the nightly's own stated mission (drift/rot over quiet periods).
Suggestion: Either (a) add `leak-scan` + `import-smoke` (and ideally `license-scan`) as nightly lanes ? they're cheap, offline, and on-mission ? or (b) soften the header comment (L8-12) to state explicitly WHICH offline lanes are intentionally in scope and why the others (leak-scan/import-smoke/license-scan/build-artifact) are deliberately excluded, so the 'mirrors verbatim' claim isn't misleading to a future reader.

Return isReal + adjustedSeverity (INVALID if not real) + one-paragraph reasoning grounded in the actual files.

2. assistant (2026-06-25T17:12:34.757Z)

I'll verify this finding by reading the actual workflow files.

3. user (2026-06-25T17:12:36.485Z)

chore/nightly-quality-regression
1a400c0 ci: add Tier-1 nightly quality-regression workflow
86705c1 chore: ruff format gemini_refine.py + trajectory_hybrid.py (#379)
b72ac85 fix(B2/P17): surface Gemini provider failures in all callers (#378)

4. user (2026-06-25T17:12:37.126Z)

```
1      name: Nightly quality regression (Tier-1, offline)
2
3      # Tier-1 nightly quality guard. Per-PR `ci.yml` only fires on push/pull_request;
4      # this runs the SAME offline guards on a schedule so we also catch what per-PR CI
5      # can't: dependency / transitive-dep drift, flakiness over time, and rot during
6      # quiet periods when nothing is pushed.
7      #
8      # Tier-1 == OFFLINE ONLY: no GPU, no real video, no `run_regression.py` on the
9      # real corpus (that's Tier-2 ? owner-gated, needs a self-hosted GPU runner +
10     # the gitignored golden corpus). The lanes here mirror `ci.yml`'s offline jobs
11     # verbatim (same Python pin, same per-lane pip installs) so modules collect/run
12     # instead of silently skipping for a missing dep.
13     #
14     # NOTE: scheduled workflows only fire from the repo's DEFAULT branch, so this
15     # won't actually run on a cadence until it lands on `master`. Use the
16     # `workflow_dispatch` trigger to run it on demand for validation after merge.
17
18     on:
19       schedule:
20         # 03:00 UTC daily.
21         - cron: "0 3 * * *"
22       workflow_dispatch:
23
24     permissions:
25       contents: read
26
27     concurrency:
28       group: nightly-`${{ github.ref }}`
29       cancel-in-progress: true
30
31     jobs:
32       # Lint gate (ruff.toml at repo root) ? mirrors ci.yml::lint.
33       lint:
34         name: Ruff lint
35         runs-on: ubuntu-latest
36         timeout-minutes: 10
37         steps:
38           - uses: actions/checkout@v6
39           - uses: actions/setup-python@v6
40             with:
41               python-version: "3.13"
42           - name: Install ruff
43             run: python -m pip install ruff
44           - name: Ruff check
45             run: ruff check . --output-format github
46
47       # Type gate (mypy.ini at repo root) ? mirrors ci.yml::typecheck. Typed core
48       # deps are installed so mypy sees real signatures (without them
49       # ignore_missing_imports would Any-stub half the checks).
50       typecheck:
51         name: Mypy (lenient baseline)
52         runs-on: ubuntu-latest
53         timeout-minutes: 10
54         steps:
55           - uses: actions/checkout@v6
56           - uses: actions/setup-python@v6
57             with:
58               python-version: "3.13"
59           - name: Install mypy + typed core deps
60             run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
61             genai numpy
62           - name: Mypy
63             run: mypy backend training
```

```

63
64 # Full offline suite + coverage floor ? mirrors ci.yml::full-suite. CPU-only
65 # torch (no CUDA wheels on a CPU runner), then the editable install pulls the
66 # rest. Floor stays at the ci.yml value (70); never lower it without an owner
67 # decision.
68 full-suite:
69   name: Full test suite (offline, mocked)
70   runs-on: ubuntu-latest
71   timeout-minutes: 30
72   steps:
73     - uses: actions/checkout@v6
74     - uses: actions/setup-python@v6
75       with:
76         python-version: "3.13"
77         cache: pip
78     - name: System libs for opencv-python
79       run: |
80         sudo apt-get update
81         sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
82         libgl1 libglu1-mesa-dev
83     - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
84       run: python -m pip install torch torchvision --index-url
85         https://download.pytorch.org/whl/cpu
86     - name: Install package (editable) + dev tooling
87       run: python -m pip install -e .[dev]
88     - name: Run full suite (with coverage floor)
89       run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
90         --cov-fail-under=70
91
92 # Offline quality guard: the committed golden regression fixtures
93 # (docs/GOLDEN_REGRESSION_FIXTURES.md) ? synthetic + de-attributed real
94 # trajectory fixtures, NO video / GPU / DB. Mirrors the offline replay in
95 # ci.yml::golden-crossos (single-OS here; ci.yml already covers cross-OS on
96 # every PR ? nightly is about drift-over-time on the default branch). The
97 # CONTROL replay needs only numpy.
98 golden-fixtures:
99   name: Golden fixtures regression (offline quality guard)
100   runs-on: ubuntu-latest
101   timeout-minutes: 15
102   steps:
103     - uses: actions/checkout@v6
104     - uses: actions/setup-python@v6
105       with:
106         python-version: "3.13"
107     - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
108       run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
109         genai numpy
110     - name: Golden-fixtures characterization (offline)
111       run: |
112         python -m backend.eval.golden_fixtures --check
113         python -m pytest tests/test_golden_fixtures.py
114         tests/test_golden_real_fixtures.py -q
115
116 # Failure visibility ? a silent nightly is useless. Scheduled runs have no PR
117 # to redden and nobody is watching the Actions tab on a quiet day (the exact
118 # "rot during quiet periods" this workflow exists to catch). On ANY guard
119 # failure, open/refresh a single tracking issue so a human is pinged. Uses the
120 # built-in GITHUB_TOKEN (no secrets); `issues: write` is scoped to THIS job
121 # only ? the guard jobs above keep `contents: read`.
122 notify-on-failure:
123   name: Surface nightly failure (open/refresh tracking issue)
124   needs: [lint, typecheck, full-suite, golden-fixtures]
125   if: failure() && github.event_name == 'schedule'
126   runs-on: ubuntu-latest
127   timeout-minutes: 5

```

```

123     permissions:
124         contents: read
125     issues: write
126     steps:
127     - uses: actions/github-script@v7
128       with:
129         script: |
130             const runUrl =
`${context.serverUrl}/${context.repo.owner}/${context.repo.repo}/actions/runs/${context.runId}`;
131             const title = 'Nightly quality regression FAILED';
132             const body = [
133                 'The Tier-1 nightly offline quality guard failed.',
134                 '',
135                 `- Run: ${runUrl}`,
136                 `- Date (UTC): ${new Date().toISOString()}`,
137                 '',
138                 'One of ruff / mypy / full-suite (cov>=70) / golden-fixtures regressed.',
139                 'Open the run for the failing lane. Close this issue once green.',
140             ].join('\n');
141             // Reuse a single open tracking issue instead of spamming one per night.
142             const existing = await github.rest.issues.listForRepo({
143                 owner: context.repo.owner,
144                 repo: context.repo.repo,
145                 state: 'open',
146                 labels: 'nightly-failure',
147                 per_page: 1,
148             });
149             if (existing.data.length > 0) {
150                 await github.rest.issues.createComment({
151                     owner: context.repo.owner,
152                     repo: context.repo.repo,
153                     issue_number: existing.data[0].number,
154                     body,
155                 });
156             } else {
157                 await github.rest.issues.create({
158                     owner: context.repo.owner,
159                     repo: context.repo.repo,
160                     title,
161                     body,
162                     labels: ['nightly-failure'],
163                 });
164             }
165

```

5. user (2026-06-25T17:12:37.363Z)

```

1     name: CI
2
3     on:
4         push:
5             branches: [master]
6         pull_request:
7
8     permissions:
9         contents: read
10
11     concurrency:
12         group: ci-${{ github.ref }}
13         cancel-in-progress: true
14
15     jobs:
16         # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17         # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18         # tracked .py, including scripts/ and training/, which the backend-only

```

```

19 # py_compile smoke sweep does not reach.
20 lint:
21     name: Ruff lint
22     runs-on: ubuntu-latest
23     steps:
24         - uses: actions/checkout@v6
25         - uses: actions/setup-python@v6
26           with:
27             python-version: "3.13"
28         - name: Install ruff
29           run: python -m pip install ruff
30         - name: Ruff check
31           run: ruff check . --output-format github
32
33 # PII / attribution leak guard (tools/leak_scan.py): fails on a committed 32-hex MD5
34 # (the video_id shape) or a personal absolute user path in the shipped-code surface
35 # (backend/training/scripts/tools/eval_baselines; tests/ excluded ? legit fixtures).
Real
36 # names are scanned only via a gitignored private list locally; CI runs the structural
guard.
37 leak-scan:
38     name: Leak scan (PII / attribution guard)
39     runs-on: ubuntu-latest
40     steps:
41         - uses: actions/checkout@v6
42         - uses: actions/setup-python@v6
43           with:
44             python-version: "3.13"
45         - name: Leak scan
46           run: python -m tools.leak_scan
47
48 # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
49 # per-module quarantine list ? ratchet by removing entries as modules clean
50 # up. Typed core deps are installed so mypy sees their real signatures
51 # (without them ignore_missing_imports would Any-stub half the checks).
52 typecheck:
53     name: Mypy (lenient baseline)
54     runs-on: ubuntu-latest
55     steps:
56         - uses: actions/checkout@v6
57         - uses: actions/setup-python@v6
58           with:
59             python-version: "3.13"
60         - name: Install mypy + typed core deps
61           run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
62         - name: Mypy
63           run: mypy backend training
64
65 # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
66 # module-level import error shipping while the suite stays green.
67 # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
68 # on machines without the GPU stack (test_import_smoke stubs what's absent).
69 import-smoke:
70     name: Syntax sweep + import smoke (no GPU stack)
71     runs-on: ubuntu-latest
72     steps:
73         - uses: actions/checkout@v6
74         - uses: actions/setup-python@v6
75           with:
76             python-version: "3.13"
77         - name: Install minimal deps
78           run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
79         - name: Run import smoke tests

```

```

80         run: python -m pytest tests/test_import_smoke.py -v
81
82     full-suite:
83         name: Full test suite (offline, mocked)
84         runs-on: ubuntu-latest
85         steps:
86             - uses: actions/checkout@v6
87             - uses: actions/setup-python@v6
88               with:
89                 python-version: "3.13"
90                 cache: pip
91             - name: System libs for opencv-python
92               run: |
93                 sudo apt-get update
94                 sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
95                 libgl1 libglu1-mesa-dev
96             - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
97               # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
98               # need an out-of-band index that PEP 621 can't express. Keep installing
99               # them separately here, before the editable install pulls the rest.
100              run: python -m pip install torch torchvision --index-url
101              https://download.pytorch.org/whl/cpu
102             - name: Install package (editable) + dev tooling
103               # Editable PEP 621 install: runtime deps + the `dev` group
104               # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
105               run: python -m pip install -e .[dev]
106             # obsreport (private, soft dep) is absent here: reporting tests skip via
107             # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
108             # is therefore calibrated against THIS lane's number (local runs with
109             # obsreport installed measure a little higher).
110             # Floor calibrated 2026-06-11 against this lane's measured 72% (local
111             # with obsreport: 73%). Ratchet it UP as coverage grows; never down
112             # without an owner decision.
113             - name: Run full suite (with coverage floor)
114               run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
115               --cov-fail-under=70
116
117     # Cross-OS determinism guard for the committed golden regression fixtures
118     # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
119     # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
120     # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
121     # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
122     # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
123     golden-crossos:
124         name: Golden fixtures cross-OS (${ matrix.os })
125         strategy:
126             fail-fast: false
127             matrix:
128                 os: [ubuntu-latest, windows-latest, macos-latest]
129         runs-on: ${ matrix.os }
130         steps:
131             - uses: actions/checkout@v6
132             - uses: actions/setup-python@v6
133               with:
134                 python-version: "3.13"
135             - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
136               run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
137               genai numpy
138             - name: Golden-fixtures characterization (cross-OS determinism guard)
139               # Also runs the freeze-real / time-origin-reset / M2 refusal-gate tests
140               (synthetic /
141               # tmp_path only ? no GPU/network) so the reset metric-invariance (fix #2) and
142               the
143               # de-attribution paths get cross-OS coverage and surface any win/mac float
144               drift.

```

```

138         run: |
139             python -m backend.eval.golden_fixtures --check
140             python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
141
142     # Commercial-clean licensing guardrail (MIT/Apache/BSD only ? PACKAGING_PLAN.md P0d).
Scans the
143     # RUNTIME dependency closure (`pip install .` ? no torch/dev) and fails on any
copyleft
144     # (GPL/AGPL/LGPL/SSPL). torch (BSD-3) is installed out-of-band and is not in this
closure. This
145     # automates a guardrail that was human-only; the report is printed for review either
way.
146     license-scan:
147         name: Dependency license scan (no copyleft in runtime deps)
148         runs-on: ubuntu-latest
149         steps:
150             - uses: actions/checkout@v6
151             - uses: actions/setup-python@v6
152               with:
153                 python-version: "3.13"
154             - name: Install runtime deps + pip-licenses
155               run: |
156                 python -m pip install .
157                 python -m pip install pip-licenses
158             - name: Report dependency licenses
159               run: python -m piplicenses --format=markdown --with-urls --order=license
160             - name: Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
161               run: |
162                 if python -m piplicenses --format=plain | grep -iE "GPL|SSPL"; then
163                     echo "::error::A runtime dependency carries a copyleft (GPL/AGPL/LGPL/SSPL)
license ? the guardrail is MIT/Apache/BSD only. Review the report above."
164                 exit 1
165             fi
166             echo "OK: no GPL/AGPL/LGPL/SSPL in the runtime dependency closure."
167
168     # Built-wheel smoke (PACKAGING_PLAN.md P0d): build the wheel, install it NON-editable
from a
169     # non-repo cwd, and launch the console script. Catches packaging regressions the
source suite
170     # can't ? namespace-package resolution (backend has no __init__.py), missing data
files, hidden
171     # imports. (Validated locally: a clean-venv wheel install + CPU torch runs `indexer
--help`.)
172     build-artifact:
173         name: Built wheel smoke (install + launch)
174         runs-on: ubuntu-latest
175         steps:
176             - uses: actions/checkout@v6
177             - uses: actions/setup-python@v6
178               with:
179                 python-version: "3.13"
180             - name: System libs for opencv-python
181               run: |
182                 sudo apt-get update
183                 sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
184             - name: Build the wheel
185               run: |
186                 python -m pip install build
187                 python -m build --wheel
188             - name: Install the built wheel + CPU torch (out-of-band index)
189               run: |
190                 python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu

```

```

191         python -m pip install dist/*.whl
192     - name: Launch the installed console script from a non-repo dir
193     run: |
194         cd /tmp
195         indexer --help
196         python -c "import backend, training, backend.utils.app_home; print('wheel
import OK')"
197

```

6. assistant (2026-06-25T17:12:44.343Z)

Both files are now fully read. Let me verify the claim about the header wording and what lanes exist in each file.

The finding is straightforward and verifiable from the files themselves. Let me confirm the specifics:

```

**ci.yml offline lanes:** `lint` (20-31), `leak-scan` (37-46), `typecheck` (52-63), `import-
smoke` (69-80), `full-suite` (82-112), `golden-crossos` (120-140), `license-scan` (146-166),
`build-artifact` (172-196). All 8 are `runs-on: ubuntu-latest`, no GPU/network ? all offline.

```

```

**nightly.yml lanes:** `lint`, `typecheck`, `full-suite`, `golden-fixtures` (mirror of golden-
crossos), plus `notify-on-failure`. So 4 guard lanes.

```

```

**Header claim (nightly.yml L10-12):** "The lanes here mirror `ci.yml`'s offline jobs verbatim
(same Python pin, same per-lane pip installs)..."

```

Let me check whether there's any in-repo justification for the omissions (e.g., a PR doc or comment explaining the scope choice).

7. user (2026-06-25T17:12:47.100Z)

```

1a400c0 ci: add Tier-1 nightly quality-regression workflow
---FILES IN PR---
.github/workflows/nightly.yml

```

8. user (2026-06-25T17:12:47.629Z)

No matches found

9. assistant (2026-06-25T17:12:58.215Z)

The PR adds only `nightly.yml`. The grep confirms `leak-scan`, `import-smoke`, `license-scan`, and `build-artifact` are not mentioned anywhere in `nightly.yml` ? neither as lanes nor as an explanatory exclusion note.

My verification confirms the factual claims:

- ci.yml has 8 jobs, all `ubuntu-latest`, all offline (no GPU, no network requirement beyond pip): `lint`, `leak-scan`, `typecheck`, `import-smoke`, `full-suite`, `golden-crossos`, `license-scan`, `build-artifact`.
- nightly.yml mirrors exactly 4: `lint`, `typecheck`, `full-suite`, `golden-fixtures` (the golden-crossos analog).
- The header (L10-12) literally says "The lanes here mirror `ci.yml`'s offline jobs verbatim" ? an unqualified universal claim, with no note that 4 offline jobs are intentionally excluded.
- `leak-scan` (`python -m tools.leak\_scan`, pure stdlib) and `import-smoke` (`pytest tests/test\_import\_smoke.py`, no GPU stack) are unambiguously offline and on-mission for a drift/rot guard. `license-scan` guards the non-negotiable MIT/Apache/BSD rule and is exactly the "transitive-dep drift" the header L4-5 names as the nightly's purpose.

The finding is accurate. It is correctly self-classified as not a runnability bug ? the 4 included lanes work. It is a documentation-fidelity/coverage gap: "mirror ... verbatim" overstates the coverage. The severity is genuinely low ? the omitted lanes still run per-PR on master pushes via ci.yml, so the only true loss is catching drift during quiet no-push periods (which is precisely the nightly's stated niche, so the gap is real but narrow). The fix is cheap

(add lanes or soften the comment). This is squarely a NIT/LOW.

10. user (2026-06-25T17:13:06.478Z)

Structured output provided successfully

11. assistant (2026-06-25T17:13:15.100Z)

Verified the finding against both workflow files on the PR branch. It is real and correctly scoped as LOW.

Key facts confirmed:

- `C:\Users\avidu\Projects\badminton-highlight-indexer\.github\workflows\ci.yml` has 8 jobs, all `ubuntu-latest` and offline: `lint`, `leak-scan`, `typecheck`, `import-smoke`, `full-suite`, `golden-crossos`, `license-scan`, `build-artifact`.
- `C:\Users\avidu\Projects\badminton-highlight-indexer\.github\workflows\nightly.yml` mirrors only 4 (`lint`, `typecheck`, `full-suite`, `golden-fixtures`) plus `notify-on-failure`.
- The nightly header (L10-12) makes an unqualified claim that the lanes "mirror `ci.yml`'s offline jobs verbatim", but a grep confirms `leak-scan`, `import-smoke`, `license-scan`, and `build-artifact` are not present anywhere in the file ? not as lanes, not as a documented exclusion.

It is genuinely a documentation-fidelity/partial-coverage gap, not a bug (the 4 included lanes are correct). The practical impact is narrow because the omitted jobs still run per-PR and on master pushes via ci.yml ? the only loss is drift-catching during quiet no-push periods, which happens to be the nightly's own stated niche. Cheap to fix (add the lanes or qualify the header). LOW is the right severity.